Štvrté cvičenie z predmetu

# Strojové učenie a neurónové siete

**Spätné šírenie chyby (backpropagation)**

Ako sa sieť učí: ktoré váhy zmeniť a o koľko

09. október 2026
I-SUNS cvičenie 4

# Kde sme skončili minule

### 1 Model

Neurónová sieť s váhami w a prahmi b. Na začiatku náhodné hodnoty.

### 2 Chyba

Kriteriálna funkcia L meria, ako veľmi sa sieť mýli (MSE, cross-entropy).

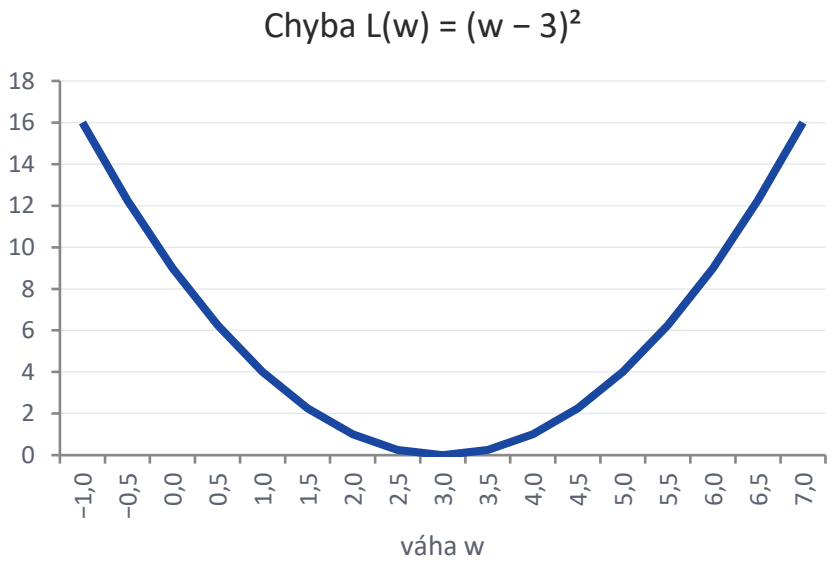
### 3 Učenie

Váhy upravujeme tak, aby L klesala: $w \leftarrow w - \alpha \cdot \text{gradient}$.

**Dnešná otázka**

**Sieť má tisíce až milióny váh. Ako zistíme, ktorú z nich a o koľko zmeniť, aby chyba klesla?**

# Gradient ukazuje smer, kam chyba rastie




Gradient = **∂L/∂w** = o koľko sa zmení L, ak váhu w trochu zvýšime.

**Pravidlo učenia**

$$w \leftarrow w - \alpha \cdot \partial L/\partial w$$

Gradient > 0: L rastie s w, preto w zmenšíme.
Gradient < 0: L klesá s w, preto w zväčšíme.
α: veľkosť kroku (learning rate).

| krok | w | gradient 2(w−3) | nové w |
|---|---|---|---|
| 0 | 0,000 | −6,000 | 1,500 |
| 1 | 1,500 | −3,000 | 2,250 |
| 2 | 2,250 | −1,500 | 2,625 |
| 3 | 2,625 | −0,750 | 2,813 |

# Tri základné brány a ich gradienty

+

**Sčítanie**

$f = x + y$

$\partial f/\partial x = 1$ $\partial f/\partial y = 1$

Rozdeľovač: gradient zhora pošle nezmenený do oboch vstupov.

×

**Násobenie**

$f = x \cdot y$

$\partial f/\partial x = y$ $\partial f/\partial y = x$

Prehadzovač: každý vstup dostane gradient zhora vynásobený hodnotou druhého vstupu.

max

**Maximum (ReLU)**

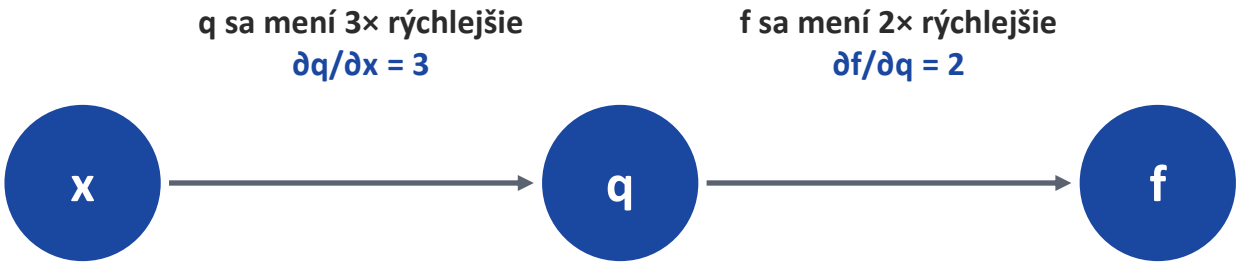
$f = \max(x, y)$

$\partial f/\partial x = 1(x \geq y)$ $\partial f/\partial y = 1(y \geq x)$

Smerovač: gradient ide len k väčšiemu vstupu, druhý dostane 0.

$1(\cdot)$ je 1, ak podmienka platí, inak 0. $\mathrm{ReLU}(x) = \max(0, x)$.

# Reťazové pravidlo: gradienty sa násobia

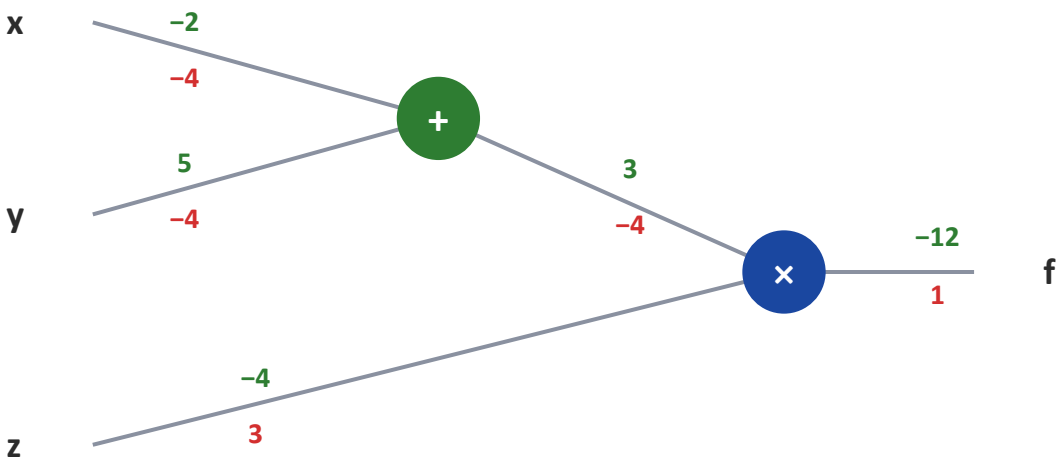



$$\partial f/\partial x = \partial f/\partial q \cdot \partial q/\partial x = 2 \cdot 3 = 6$$

**Pravidlo pre každú bránu pri spätnom šírení**

**gradient vstupu = lokálny gradient brány × gradient prichádzajúci zhora (z výstupu)**

# Príklad 1: f(x, y, z) = (x + y) · z

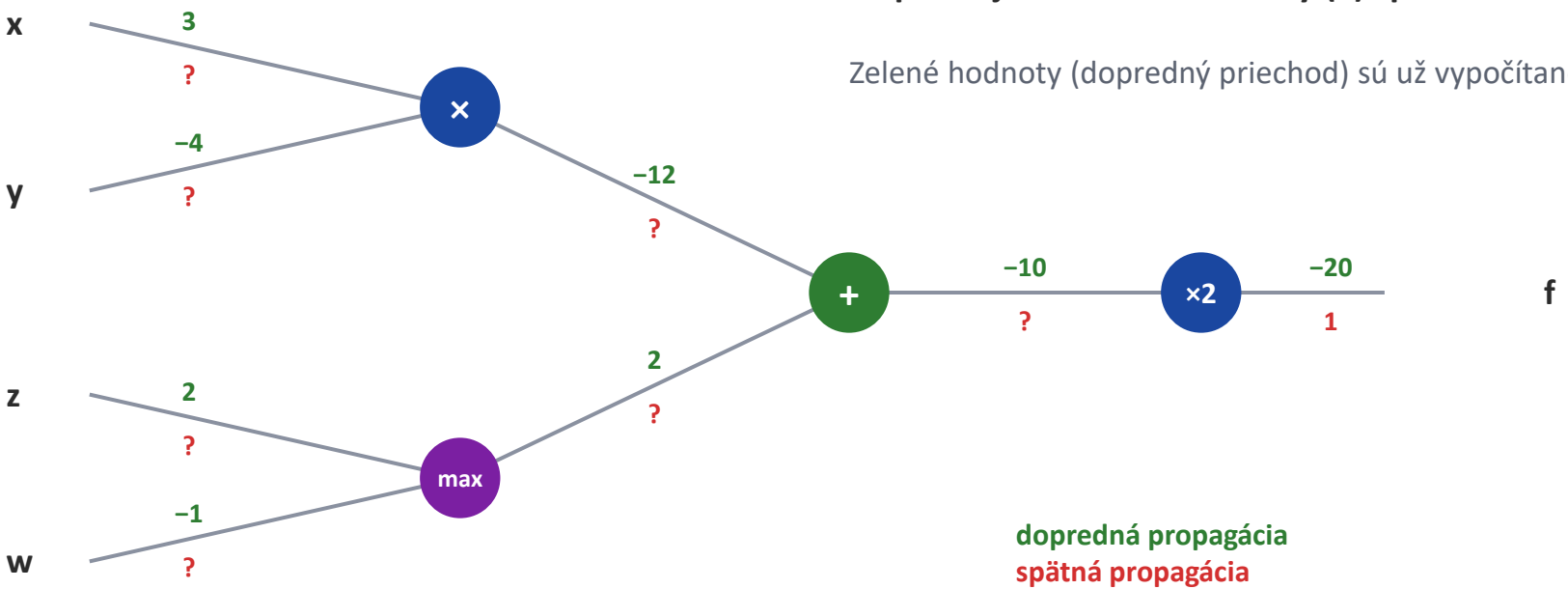



**dopredná propagácia**
**spätná propagácia**

## Krok za krokom

1. Dopredu: q = x + y = 3, f = q·z = −12
2. Na výstupe: ∂f/∂f = 1
3. Brána ×: ∂f/∂q = z = −4, ∂f/∂z = q = 3
4. Brána +: ∂f/∂x = ∂f/∂y = −4 · 1 = −4

# Príklad 2: f = (x·y + max(z, w)) · 2

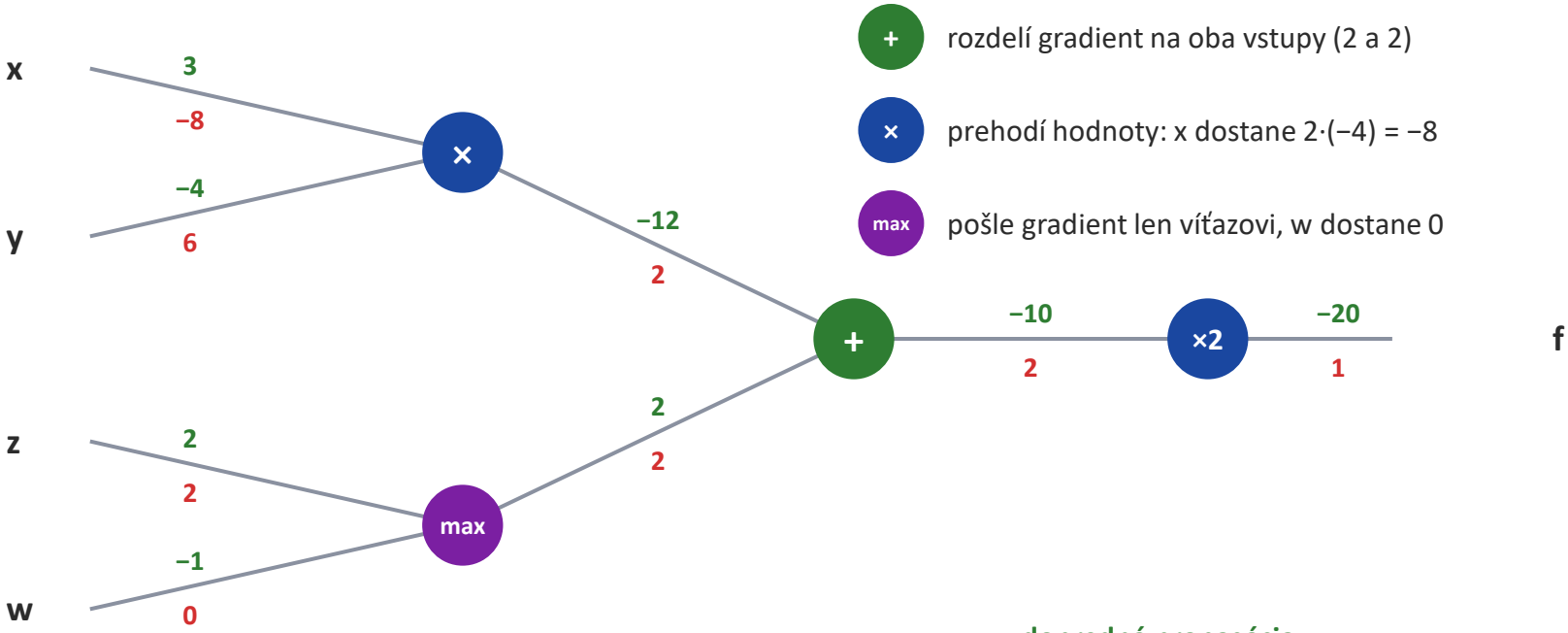



**Dopočítajte červené hodnoty (?) zprava doľava.**

Zelené hodnoty (dopredný priechod) sú už vypočítané.

**dopredná propagácia**
**spätná propagácia**

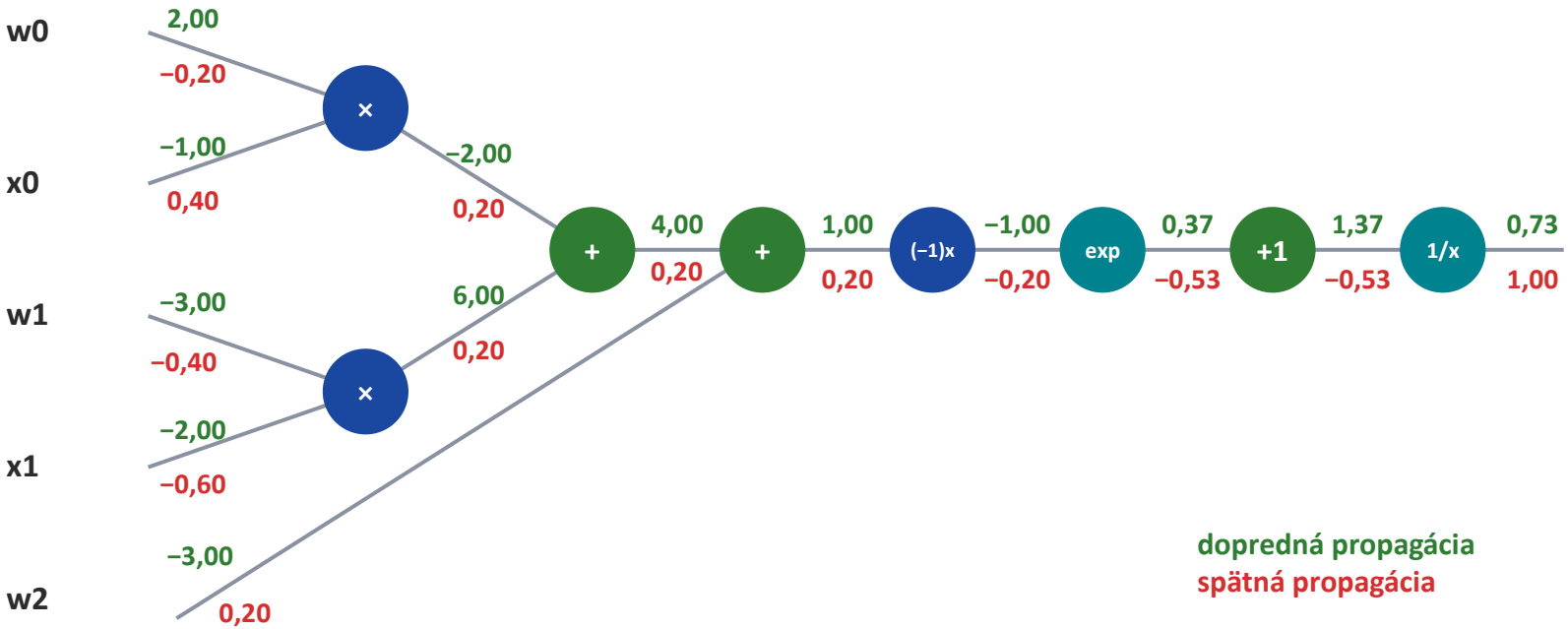
# Riešenie


x
y
z
w
3
−8
−4
6
2
2
−1
0
×
max
−12
2
2
2
+
−10
2
×2
−20
1
f
+ rozdelí gradient na oba vstupy (2 a 2)
× prehodí hodnoty: x dostane 2·(−4) = −8
max pošle gradient len víťazovi, w dostane 0
dopredná propagácia
spätná propagácia

# Príklad 3: neurón so sigmoidou

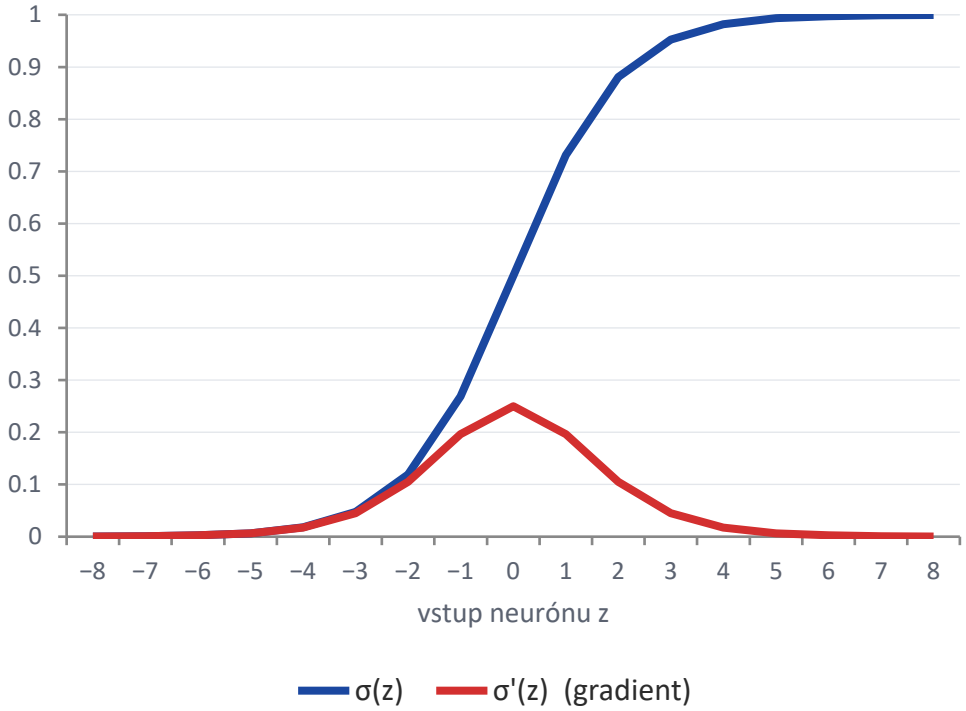
**f(w, x) = 1 / (1 + e^–(w0·x0 + w1·x1 + w2))**

Zelené čísla dostaneme zľava doprava, červené sprava doľava.



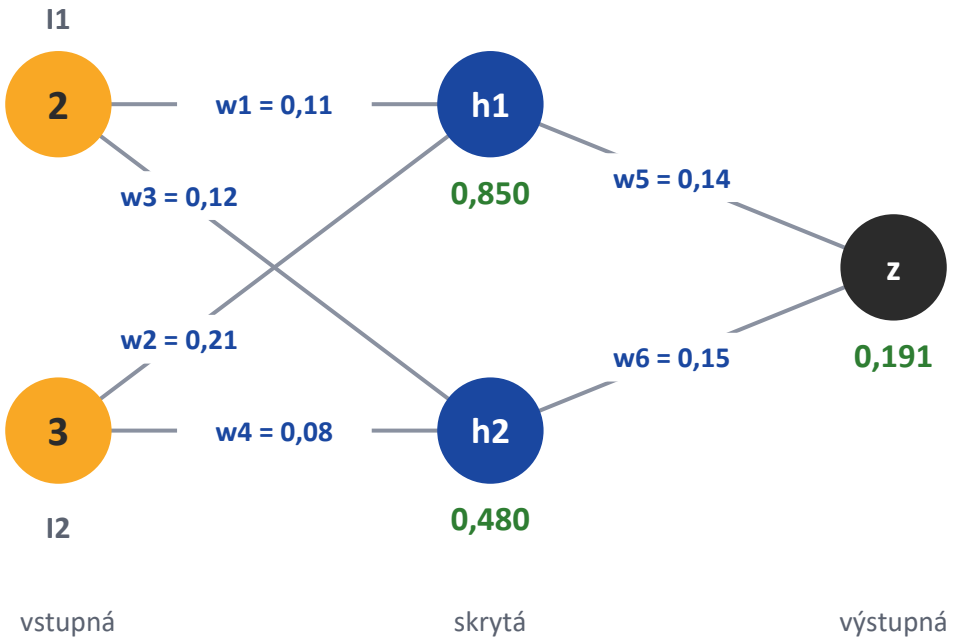

**dopredná propagácia**
**spätná propagácia**

# Pozor: sigmoida a miznúci gradient




- Derivácia sigmoidy je $\sigma(z)\cdot(1-\sigma(z)) = 0{,}73 \cdot 0{,}27 \approx 0{,}20$ ako v grafe; najviac 0,25 (pri $z = 0$), pre veľké $|z|$ ide k 0.
- Každá sigmoidová vrstva gradient vynásobí číslom ≤ 0,25. Po 10 vrstvách zostane $\approx 0{,}25^{10} \approx 10^{-6}$.
- Prvé vrstvy sa takmer neučia: miznúci gradient (vanishing gradient).
- **Riešenie: ReLU v skrytých vrstvách (derivácia 1 pre x > 0).**

# Príklad 4: malá sieť 2–2–1 (dopredu)




## Zadanie

Vstupy I1 = 2, I2 = 3

Skutočný výstup y = 1

Aktivácie lineárne (zjednodušenie)

Chyba $E = \frac{1}{2}(z - y)^2$

$\alpha = 0{,}05$

**Dopredu**

$h1 = 2 \cdot 0{,}11 + 3 \cdot 0{,}21 = 0{,}85$

$h2 = 2 \cdot 0{,}12 + 3 \cdot 0{,}08 = 0{,}48$

$z = 0{,}85 \cdot 0{,}14 + 0{,}48 \cdot 0{,}15 = 0{,}191$

**$E = \frac{1}{2}(0{,}191 - 1)^2 = 0{,}327$**

# Príklad 4: gradienty výstupnej vrstvy

**1** **Chyba voči výstupu**

$E = \frac{1}{2}(z - y)^2$

$\partial E/\partial z = z - y = \Delta = -0{,}809$

**2** **Z výstupu do váhy**

$z = h1 \cdot w5 + h2 \cdot w6$

$\partial z/\partial w5 = h1 = 0{,}85$
$\partial z/\partial w6 = h2 = 0{,}48$

**3** **Reťazové pravidlo**

násobíme lokálne gradienty

$\partial E/\partial w5 = \Delta \cdot h1 = -0{,}688$
$\partial E/\partial w6 = \Delta \cdot h2 = -0{,}388$

**Úprava váh: $w = w - \alpha \cdot \text{gradient}$**

$w5 = 0{,}14 - 0{,}05 \cdot (-0{,}688) = 0{,}1744$ $w6 = 0{,}15 - 0{,}05 \cdot (-0{,}388) = 0{,}1694$

Gradient je záporný, predikcia z je príliš nízka, preto váhy rastú.

# Príklad 4: gradienty skrytej vrstvy

| Váha | Cesta od chyby | Gradient | Nová váha |
|---|---|---|---|
| w1 = 0,11 | ∂E/∂w1 = Δ · w5 · I1 | −0,2265 | 0,1213 |
| w2 = 0,21 | ∂E/∂w2 = Δ · w5 · I2 | −0,3398 | 0,2270 |
| w3 = 0,12 | ∂E/∂w3 = Δ · w6 · I1 | −0,2427 | 0,1321 |
| w4 = 0,08 | ∂E/∂w4 = Δ · w6 · I2 | −0,3640 | 0,0982 |

**Prečo w5 a I1?**

w1 ovplyvní h1, ten ovplyvní z, z ovplyvní E. Gradient je súčin troch lokálnych gradientov: ∂E/∂z · ∂z/∂h1 · ∂h1/∂w1 = Δ · w5 · I1.

**Po jednom kroku**

h1 = 0,924

h2 = 0,559

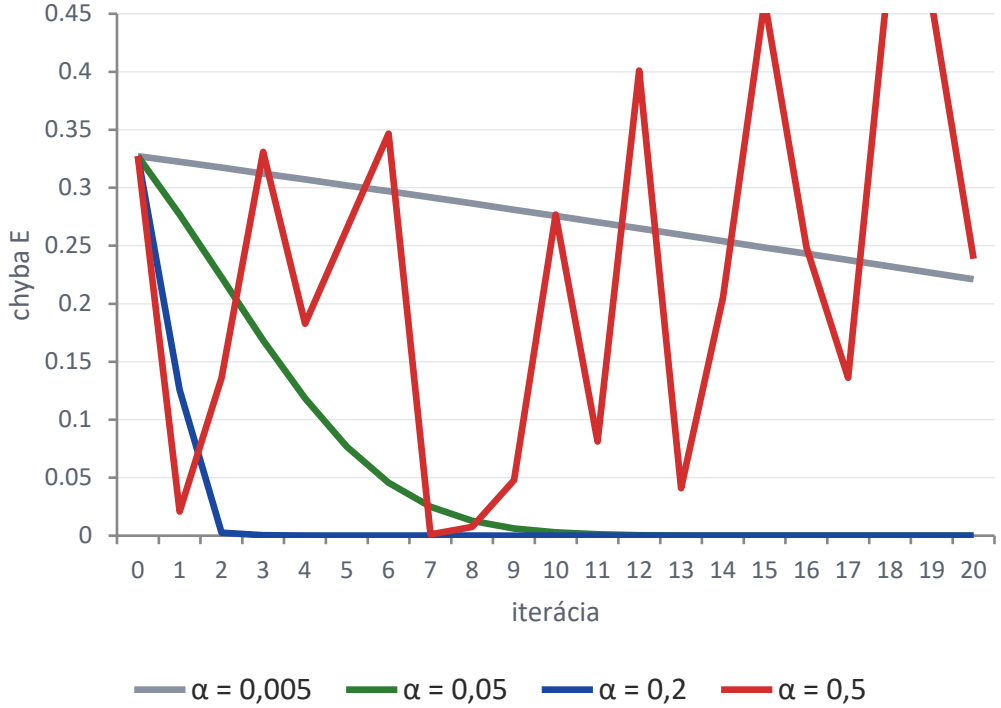
z = 0,256 (bolo 0,191)

**E = 0,277**

(bolo 0,327)

**Pozor: gradienty počítame so starými váhami, až potom aktualizujeme všetky naraz.**

# Jeden krok nestačí: iterujeme




## Prečo iterujeme?

- Gradient platí len v okolí aktuálnych váh, preto robíme malé kroky.
- Po každom kroku sa gradient zmení, musíme ho prepočítať.

## Vplyv α

- príliš malé: učenie je pomalé
- príliš veľké: chyba kmitá alebo exploduje

# Na čo si dať pozor pri trénovaní

| Čo vidíme | Pravdepodobná príčina | Čo skúsiť |
|---|---|---|
| Chyba klesá veľmi pomaly alebo vôbec | $\alpha$ príliš malé; miznúci gradient (sigmoid/tanh v hlbokej sieti); nenormalizované vstupy | zvýšiť $\alpha$, ReLU, Adam, normalizovať vstupy |
| Chyba kmitá, rastie alebo je NaN | $\alpha$ príliš veľké; explodujúci gradient | znížiť $\alpha$, normalizovať vstupy, orezať gradient |
| Trénovacia chyba klesá, validačná rastie | pretrénovanie | early stopping, viac dát, menšia sieť, regularizácia |
| Obe chyby zostávajú vysoké | podtrénovanie | väčšia sieť, viac epoch, lepšie príznaky |
| Sieť dáva stále rovnaký výstup | chyba v dátach či značkách, zlá kombinácia výstupu a chyby, mŕtve ReLU | skontrolovať dáta, aktiváciu a chybu, znížiť $\alpha$ |

**Rýchle kontroly pred dlhým trénovaním**

- Počiatočná chyba pri K triedach (softmax + cross-entropy) ≈ ln K, pre 3 triedy ≈ 1,10.
- Sieť musí vedieť „zapamätať“ 10 až 20 príkladov takmer na nulovú chybu.

# Súhrn: ako sa trénuje neurónová sieť

**1 Dáta** — CVI 3

rozdelenie train / validácia / test, normalizácia vstupov

**2 Architektúra** — CVI 3

vrstvy, neuróny, aktivácie (skryté ReLU, výstup podľa úlohy)

**3 Kriteriálna funkcia** — CVI 3

MSE pre regresiu, cross-entropy pre klasifikáciu

**4 Dopredná propagácia** — CVI 3 · 4

predikcia z a hodnota chyby L

**5 Spätná propagácia** — CVI 4

gradient L voči každej váhe cez reťazové pravidlo

**6 Solver a α** — CVI 3 · 4

SGD alebo Adam: $w \leftarrow w - \alpha \cdot \text{gradient}$

**7 Opakovanie** — CVI 3

dávky, epochy, zastavovacia podmienka (early stopping)

**8 Sledovanie** — CVI 3

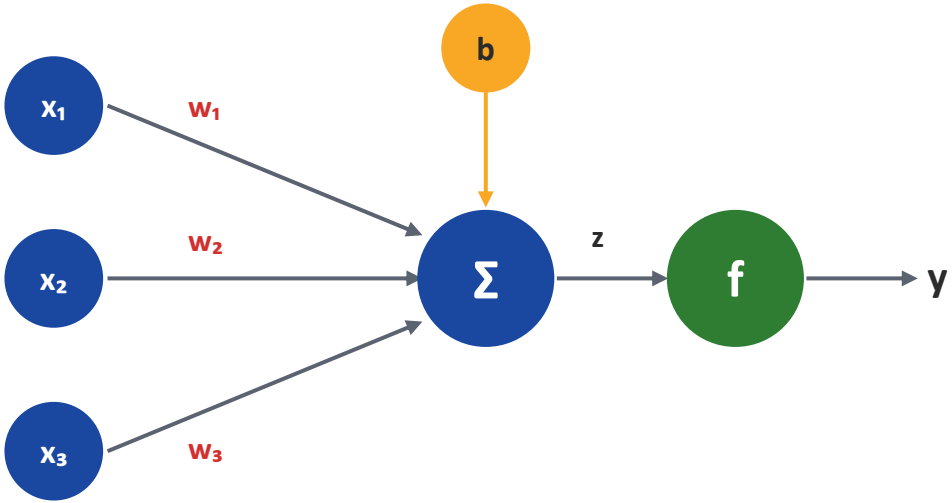
krivky chyby na trénovacej a validačnej množine, test na konci

Dnes: kroky 4 a 5 (červená) sme prešli ručne na troch príkladoch.

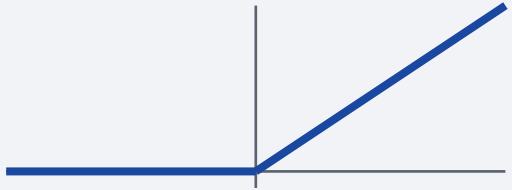
# Jeden neurón: vstupy, váhy, bias, aktivácia

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$$

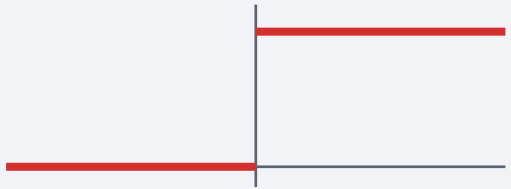
$$y = f(z)$$ f = aktivačná funkcia




**ReLU: f(z) = max(0, z)**



**Prahová: f(z) = 1 ak z ≥ 0, inak 0**



Pri učení sa používa hladká sigmoida; prahová funkcia je len ukážka najjednoduchšieho neurónu.

# Jeden neurón: vypočítaný príklad

| vstup | x | váha w | x · w |
|---|---|---|---|
| $x_1$ | 2 | 0,5 | 1 |
| $x_2$ | −1 | −1 | 1 |
| $x_3$ | 3 | 0,2 | 0,6 |
| **súčet Σ x·w** | | | **2,6** |
| bias b | | | **−1,2** |
| **z = Σ x·w + b** | | | **1,4** |

Pri učení sa menia práve váhy w a bias b.

**ReLU**

**max(0; 1,4) = 1,4**

**Prahová**

**1,4 ≥ 0 → 1**

**Čo urobí bias?**

Pri b = −3 vyjde z = −0,4, takže ReLU aj prahová funkcia dajú 0: neurón sa „nezapne“.